

The lazy programmer's guide to writing 1000's of tests

An introduction to property based testing

Part I:

In which I have a conversation with a
remote developer

This was a project from a long time ago,
in a galaxy far far away

For some reason
we needed a
custom "add"
function

First, I had a look at the existing tests...

```
[<Test>]  
let ``When I add 1 + 3, I expect 4``()=  
    let result = add 1 3  
    Assert.AreEqual(4,result)
```



```
[<Test>]  
let ``When I add 2 + 2, I expect 4``()=  
    let result = add 2 2  
    Assert.AreEqual(4,result)
```



Ok, now for my first new test...

```
[<Test>]  
let ``When I add -1 + 3, I expect 2``()=  
    let result = add -1 3  
    Assert.AreEqual(2,result)
```



That's funny..

Hmm.. let's look at the implementation...

```
let add x y =  
  4
```

wtf!

Time for some more tests...

```
[<Test>]  
let ``When I add 2 + 3, I expect 5``()=  
    let result = add 2 3  
    Assert.AreEqual(5,result)
```



```
[<Test>]  
let ``When I add 1 + 41, I expect 42``()=  
    let result = add 1 41  
    Assert.AreEqual(42,result)
```



Let's just check the implementation again...

```
let add x y =  
  match (x,y) with  
  | (2,3) -> 5  
  | (1,41) -> 42  
  | (_,_) -> 4    // all other cases
```

wtf wtf wtf

TDD best practices

Write the minimal code that will make the test pass

At this point you need to write code that will successfully pass the test.

The code written at this stage will **not** be 100% final, you will improve it later stages.

Do not try to write the perfect code at this stage, just write code that will pass the test.

Another attempt at a test

```
[<Test>]
let ``When I add two numbers,
    I expect to get their sum``()=

    for (x,y,expected) in [
        (1,2,3);
        (2,2,4);
        (3,5,8);
        (27,15,42); ]

    let actual = add x y
    Assert.AreEqual(expected,actual)
```



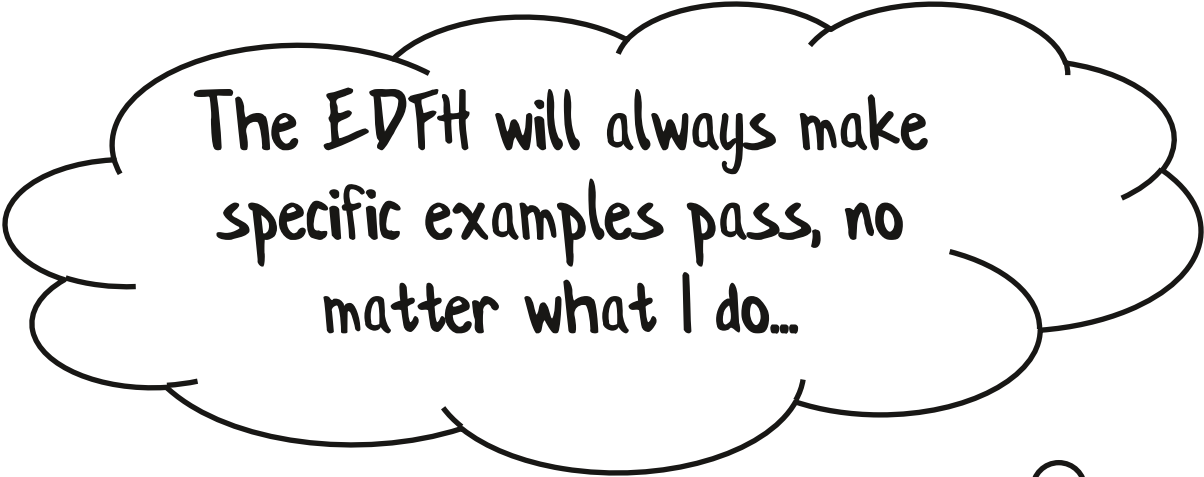
Let's check the implementation one more time....

```
let add x y =  
  match (x,y) with  
  | (1,2) -> 3  
  | (2,3) -> 5  
  | (3,5) -> 8  
  | (1,41) -> 42  
  | (25,15) -> 42  
  | (_,_) -> 4    // all other cases
```

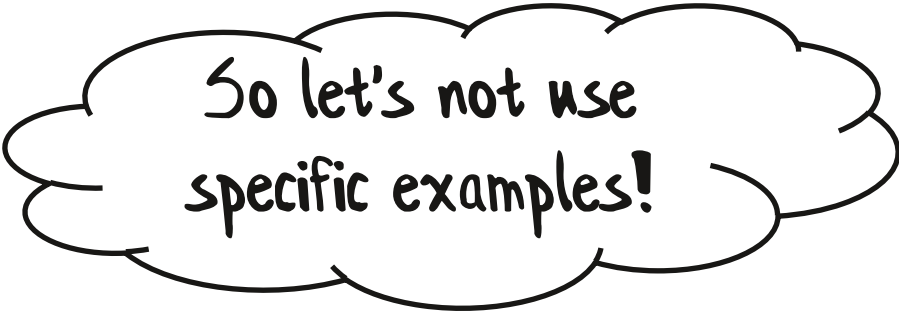
It dawned on me who I was
dealing with...

...the legendary burned-out, always lazy and
often malicious programmer called...

Rethinking the approach



The EDFH will always make
specific examples pass, no
matter what I do...



So let's not use
specific examples!

Let's use random numbers instead...

```
[<Test>]
```

```
let ``When I add two random numbers,  
    I expect their sum to be correct``=
```

```
    let x = randInt()
```

```
    let y = randInt()
```

```
    let expected = x + y
```

```
    let actual = add x y
```

```
    Assert.AreEqual(expected, actual)
```

And why not do it 100 times just to be sure...

```
[<Test>]
let ``When I add two random numbers (100 times),
    I expect their sum to be correct``()=

for _ in [1..100] do
  let x = randInt()
  let y = randInt()
  let expected = x + y
  let actual = add x y
  Assert.AreEqual(expected, actual)
```

Yea! Problem solved!

The EDFH can't beat this!

```
[<Test>]
let ``When I add two random numbers (100 times),
    I expect their sum to be correct``()=

for _ in [1..100] do
  let x = randInt()
  let y = randInt()
  let expected = x + y ← Uh-oh!
  let actual = add x y
  Assert.AreEqual(expected, actual)
```

We can't test "add" using +!

But if you can't test by using +, how CAN you test?

Part II:

Property based testing

What are the "requirements" for the "add" function?

It's hard to know where to get started, but one approach is to compare it with something different...

How does "add" differ from "subtract", for example?

So how does "add" differ from "subtract"?

For "subtract", the order of the parameters makes a difference, while for "add" it doesn't.

```
[<Test>]
let ``When I add two numbers, the result
    should not depend on parameter order``()=

for _ in [1..100] do
  let x = randInt()
  let y = randInt()
  let result1 = add x y
  let result2 = add y x ← reversed params
  Assert.AreEqual(result1, result2)
```

It doesn't depend on addition, and it eliminates a whole class of incorrect implementations!

The EDFH responds with:

```
let add x y =  
  x * y
```

TEST: ``When I add two numbers, the result
should not depend on parameter order``



Ok, what's the difference
between add and multiply?

Example: two "add 1"s is the same as one "add 2".

Test: two "add 1"s is the same as one "add 2".

```
[<Test>]
let ``Adding 1 twice is the same as adding 2``()=

  for _ in [1..100] do
    let x = randInt()
    let y = randInt()
    let result1 = x |> add 1 |> add 1
    let result2 = x |> add 2
    Assert.AreEqual(result1,result2)
```

The EDFH responds with:

```
let add x y =  
  x - y
```

TEST: ``Adding 1 twice is the same as adding 2``



But luckily we have the previous test as well!

TEST: ``When I add two numbers, the result
should not depend on parameter order``



Ha! Gotcha, EDFH!

The EDFH responds with another implementation:

```
let add x y =  
  0
```

TEST: ``Adding 1 twice is the same as adding 2``



TEST: ``When I add two numbers, the result should not depend on parameter order``



Aarrghh! Where did our approach go wrong?

We have to check that the result is somehow connected to the input.

Is there a trivial property of add that we know the answer to without reimplementing our own version?

Yes! Adding zero is the same as doing nothing

```
[<Test>]
let ``Adding zero is the same as doing nothing``()=

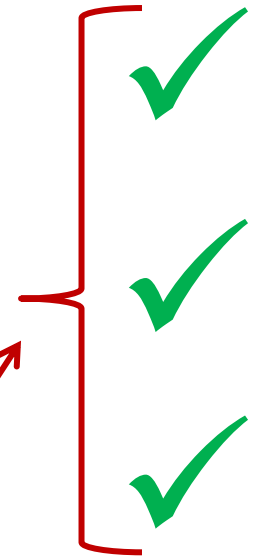
  for _ in [1..100] do
    let x = randInt()
    let result1 = x |> add 0
    let result2 = x
    Assert.AreEqual(result1,result2)
```

Finally, the EDFH is defeated...

TEST: ``Adding 1 twice is the same as adding 2``

TEST: ``When I add two numbers, the result
should not depend on parameter order``

TEST: ``Adding zero is the same as doing nothing``



If these are all true we
MUST have a correct
implementation*

* not quite true

Refactoring

Let's extract the shared code...

Pass in a "property"

```
let propertyCheck property =  
  // property has type: int -> int -> bool  
  
  for _ in [1..100] do  
    let x = randInt()  
    let y = randInt()  
    let result = property x y  
    Assert.IsTrue(result)
```

Check the property is
true for random inputs

And the tests now look like:

```
let commutativeProperty x y =  
  let result1 = add x y  
  let result2 = add y x  
  result1 = result2
```

```
[<Test>]  
let ``When I add two numbers, the result  
    should not depend on parameter order``=  
  
  propertyCheck commutativeProperty
```

And the second property

```
let adding1TwiceIsAdding2OnceProperty x _ =  
  let result1 = x |> add 1 |> add 1  
  let result2 = x |> add 2  
  result1 = result2
```

```
[<Test>]  
let ``Adding 1 twice is the same as adding 2``()=  
  propertyCheck adding1TwiceIsAdding2OnceProperty
```

This is really just a crude
version of associativity!

And the third property

```
let identityProperty x _ =  
  let result1 = x |> add 0  
  result1 = x
```

```
[<Test>]  
let ``Adding zero is the same as doing nothing``()=  
  propertyCheck identityProperty
```

Review

Testing with properties

- The parameter order doesn't matter
- Doing "add 1" twice is the same as doing "add 2" once
- Adding zero does nothing

These properties
apply to ALL inputs
So we have a very
high confidence that
the implementation is
correct

^{Specification} Testing with properties

- "Commutativity" property
- "Associativity" property
- "Identity" property

These properties
define addition!

The EDFH can't create an
incorrect implementation!

Bonus: By using specifications, we have
understood the requirements in a deeper way.

Why bother with the EDFH?

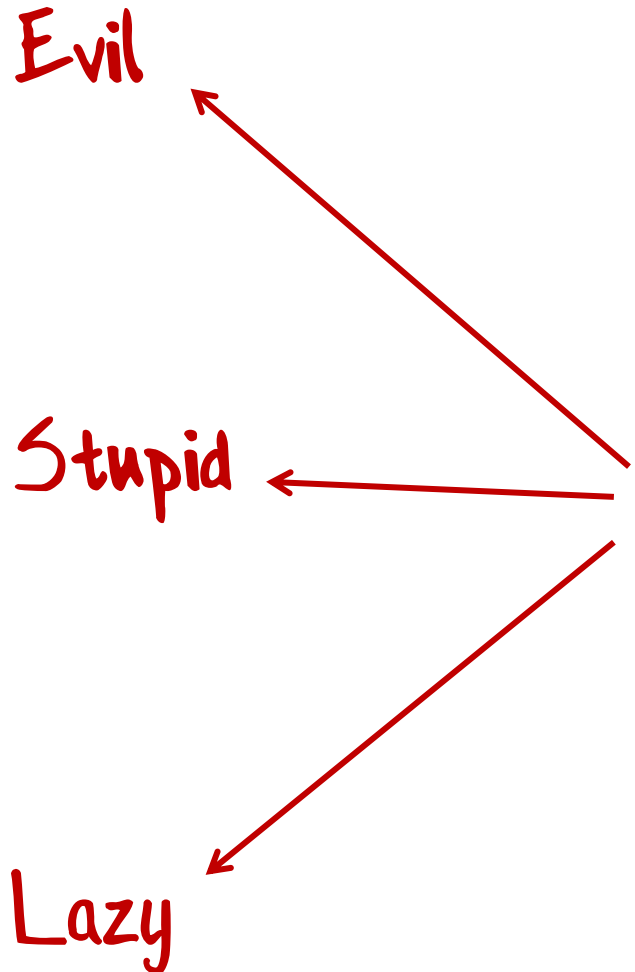
Surely such a malicious programmer is
unrealistic and over-the-top?

Evil

Stupid

Lazy

In practice,
no difference!



```
graph LR; A["In practice, no difference!"] --> B[Evil]; A --> C[Stupid]; A --> D[Lazy];
```

In my career, I've always had to deal with one
stupid person in particular ☹️

Me!

The real EDFH!

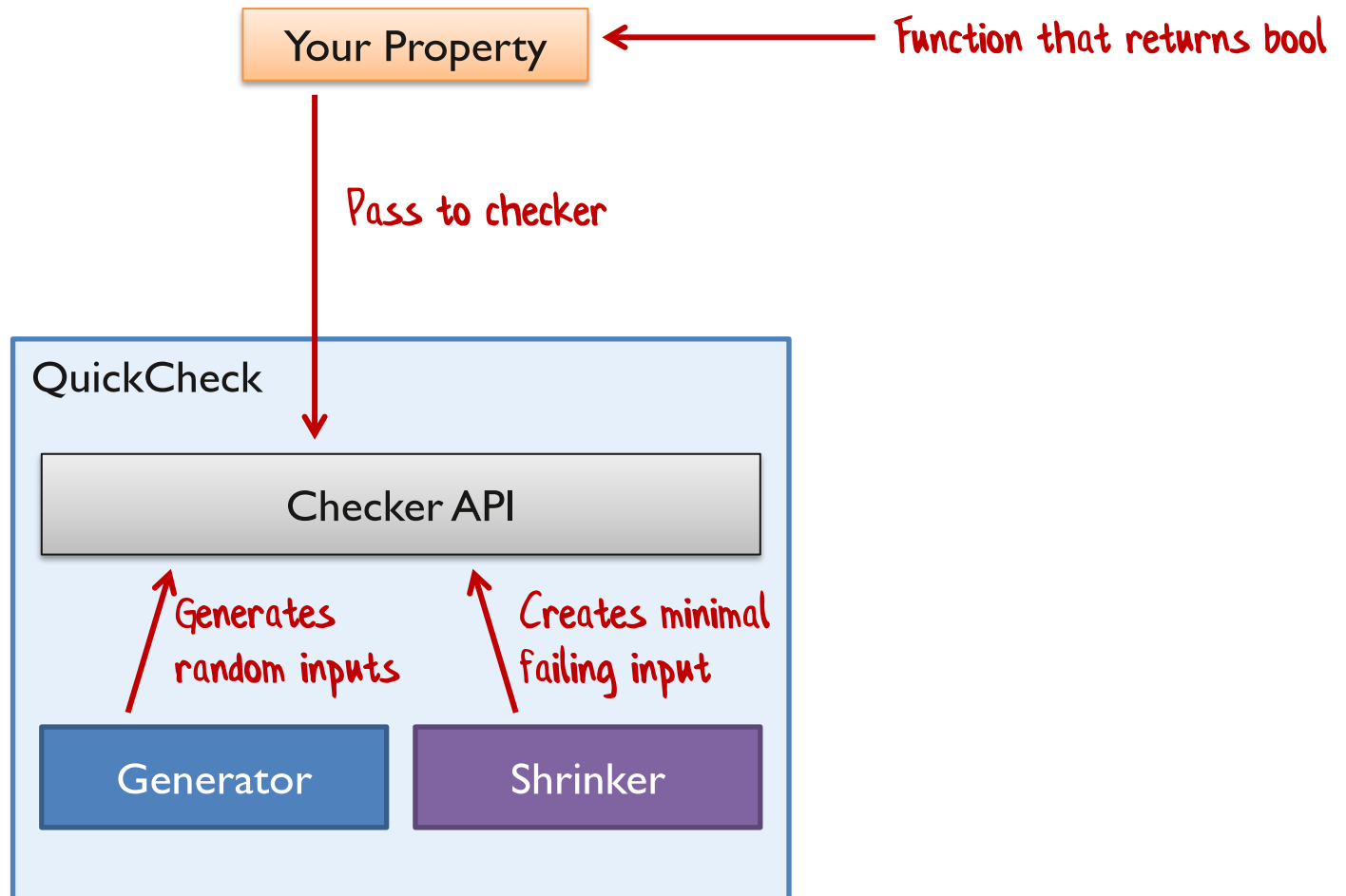
When I look at my old code, I almost always see something wrong!
I've often created flawed implementations, not out of evil
intent, but out of unawareness and blindness

Part III:

QuickCheck and its ilk

Wouldn't it be nice to have a toolkit for doing this?

The "QuickCheck" library was originally developed for Haskell by Koen Claessen and John Hughes, and has been ported to many other languages.



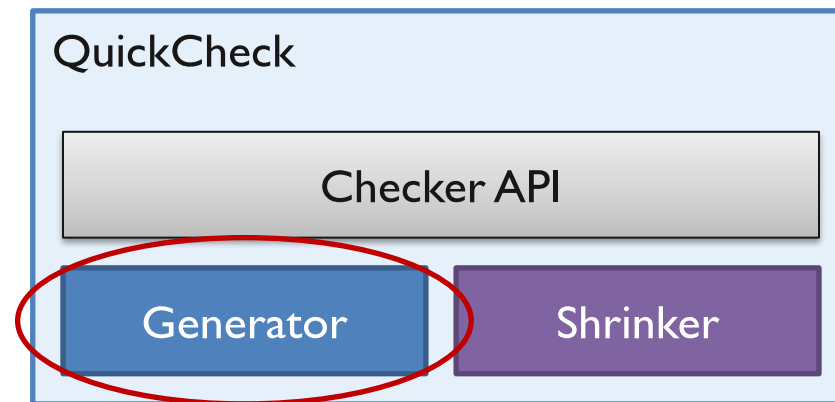
Using QuickCheck (FsCheck) looks like this:

```
// correct implementation of add!  
let add x y = x + y  
  
let commutativeProperty x y =  
  let result1 = add x y  
  let result2 = add y x  
  result1 = result2  
  
// check the property interactively  
Check.Quick commutativeProperty
```

And get the output:

```
Ok, passed 100 tests.
```

Generators: making random inputs



Generating primitive types

Generates ints
↓

"int" generator

0, 1, 3, -2, ... etc

Generates strings
↓

"string" generator

"", "eiX\$a^", "U%0Ika&r", ... etc

Generates bools
↓

"bool" generator

true, false, false, true, ... etc

Generating compound types

Generates pairs of ints



"int*int" generator

(0,0), (1,0), (2,0), (-1,1), (-1,2) ... etc

Generates options



"int option" generator

Some 0, Some -1, None, Some -4; None ...

Define custom type



```
type Color = Red | Green of int | Blue of bool
```

"Color" generator

Green 47, Red, Blue true, Green -12, ...

Generates values of custom type



Algebraic data types are the perfect partner for PBT because new types are composed from smaller ones

How it works in practice

```
let commutativeProperty (x,y) =  
  let result1 = add x y  
  let result2 = add y x // reversed params  
  result1 = result2
```

(a) Checker detects that the input is a pair of ints

Checker API

(b) Appropriate generator will be automatically created

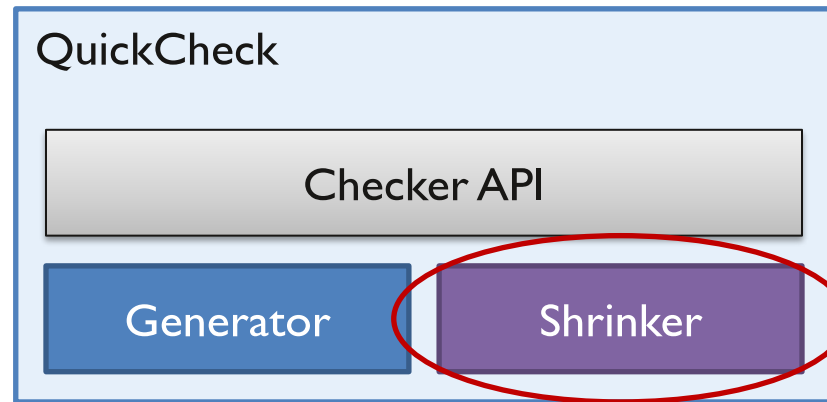
int*int generator

(c) Valid values will be generated...

(0,0) (1,0) (2,0) (-1,1) (100,-99) ...

(d) ...and passed to the property for evaluation

Shrinking: dealing with failure



How shrinking works

Property to test – we know it's gonna fail!

```
let smallerThan81Property x =  
  x < 81
```

"int" generator

0, 1, 3, -2, 34, -65, 100

Fails at 100!

So 100 fails, but knowing that is not very helpful

Time to start shrinking!

How shrinking works

```
let smallerThan81Property x =  
  x < 81
```

Shrink list for 100

0, 50, 75, 88, 94, 97, 99

Fails at 88!

Generate a new
sequence up to 100

Shrink again starting at 88

How shrinking works

```
let smallerThan81Property x =  
  x < 81
```

Shrink list for 88

0, 44, 66, 77, 83, 86, 87

Fails at 83!

Generate a new
sequence up to 88

Shrink again starting at 83

How shrinking works

```
let smallerThan81Property x =  
  x < 81
```

Shrink list for 83

0, 42, 63, 73, 78, 81, 82

Fails at 81!

Generate a new
sequence up to 83

Shrink again starting at 81

How shrinking works

```
let smallerThan81Property x =  
  x < 81
```

Shrink list for 81

0, 41, 61, 71, 76, 79, 80

Generate a new
sequence up to 81



All pass!



Shrink has determined that 81 is
the smallest failing input!

Shrinking – final result

Shrinking is built into the check:

```
Check.Quick smallerThan81Property
```

```
// result: Falsifiable, after 23 tests (3 shrinks)  
// 81
```

Shrinking is really helpful to show
the boundaries where errors happen

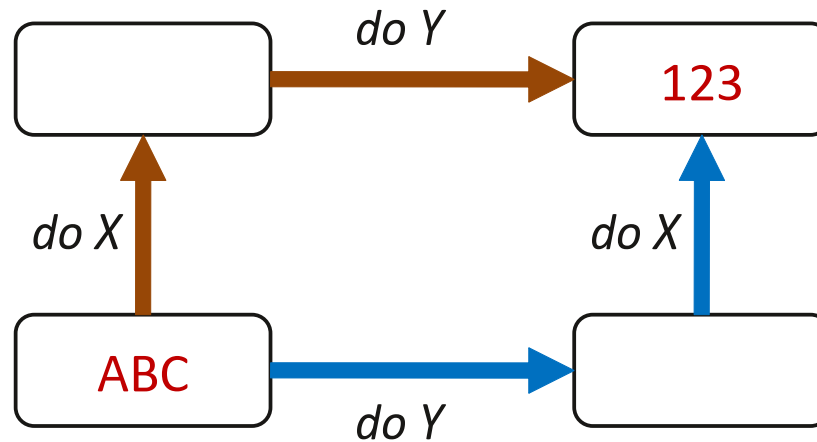
Shrinking works with
compound types too!

Part IV:

How to choose properties

What properties should I use? I can't think of any!

"Different paths, same destination"

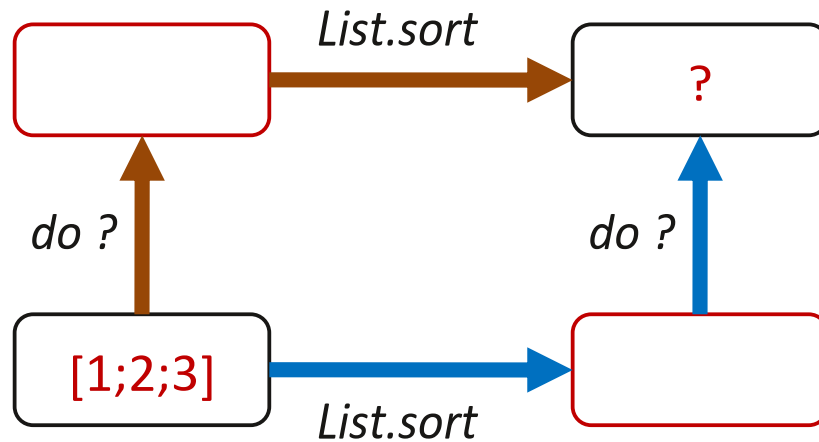


Examples:

- Commutativity
- Associativity
- Map
- Monad & Functor laws

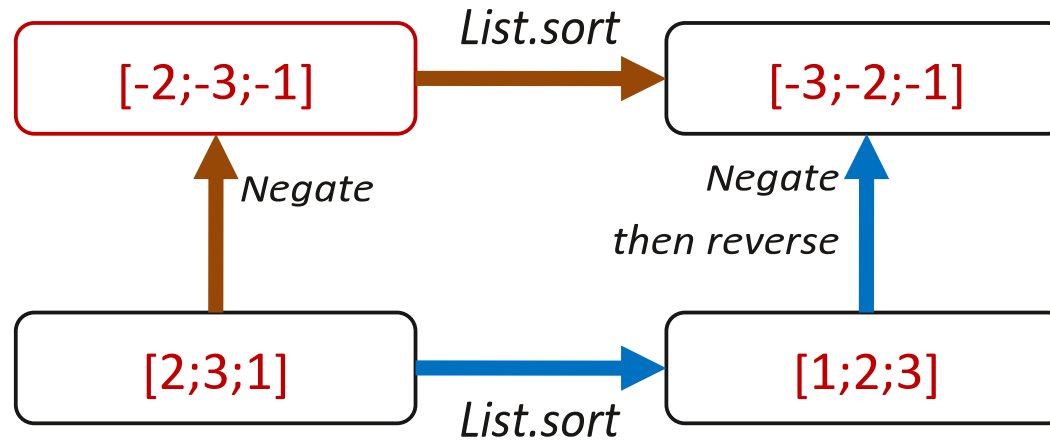
"Different paths, same destination"

Applied to a sort function



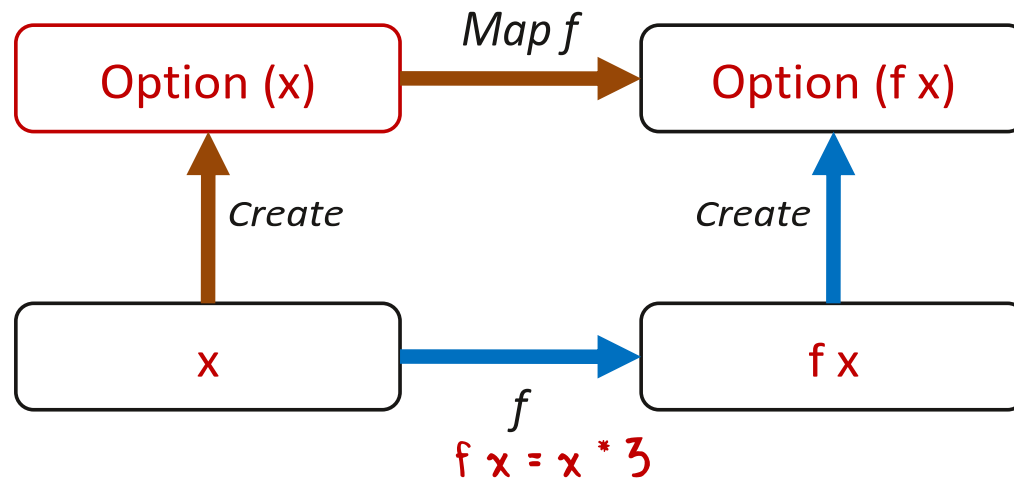
"Different paths, same destination"

Applied to a sort function



"Different paths, same destination"

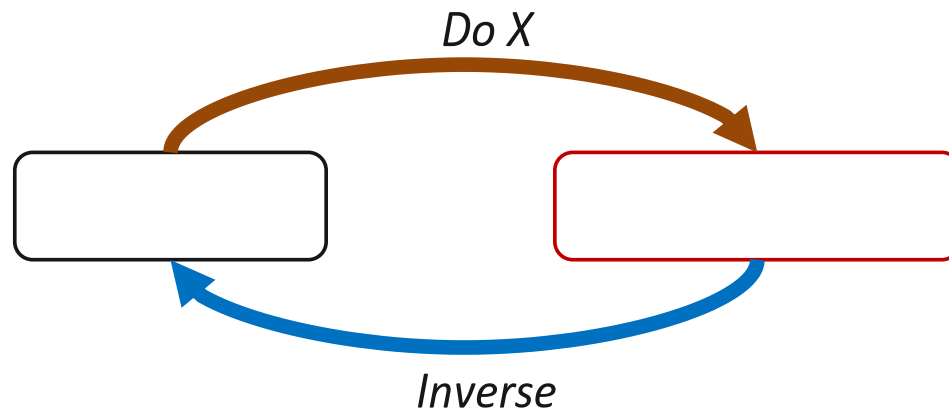
Applied to a map function



`Some(2)`
`.Map(x => x * 3)`

`Some(2 * 3)`

"There and back again"

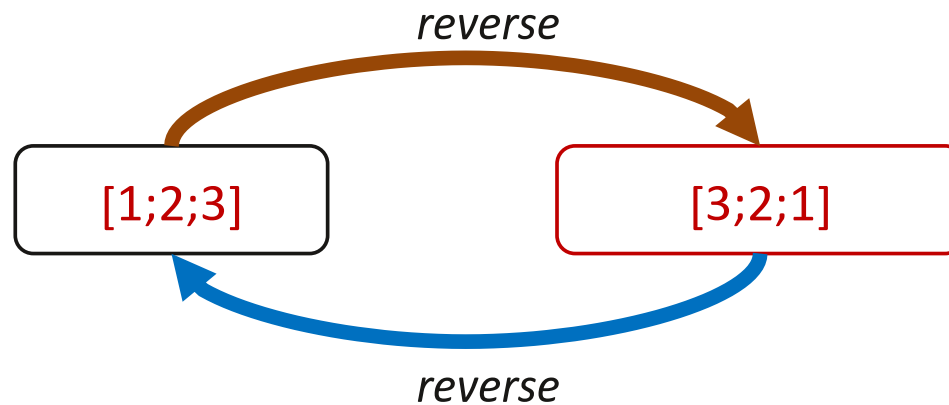


Examples:

- Serialization/Deserialization
- Addition/Subtraction
- Write/Read
- SetProperty/GetProperty

"There and back again"

Applied to a list reverse function



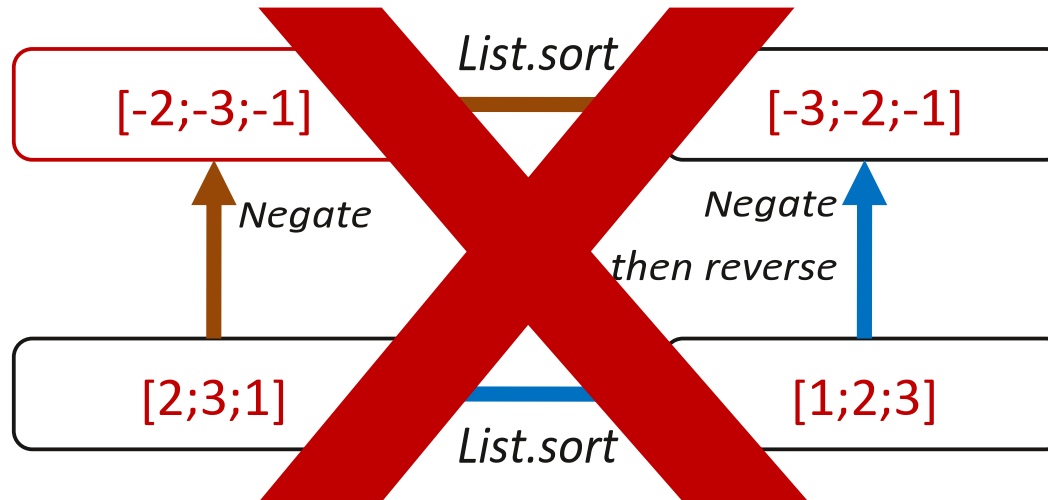
"Some things never change"



Examples:

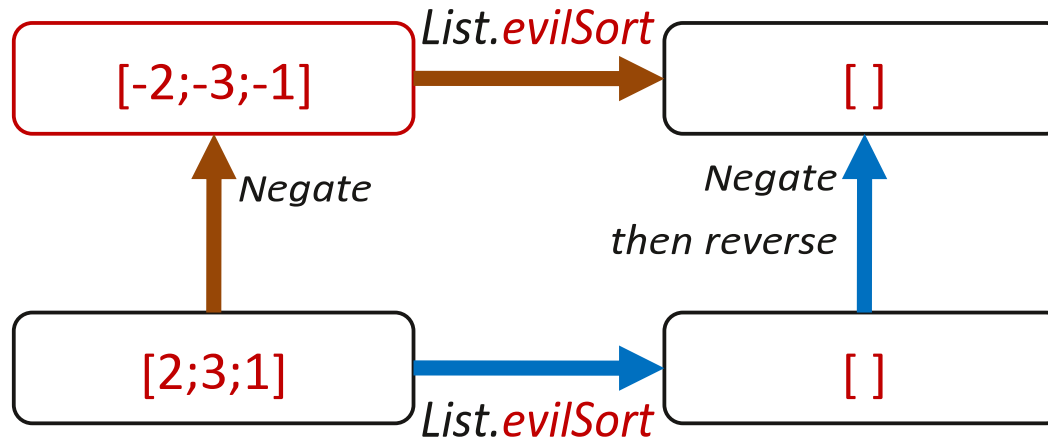
- Size of a collection
- Contents of a collection
- Balanced trees

The EDFH and List.Sort



The EDFH can beat this!

The EDFH and List.Sort

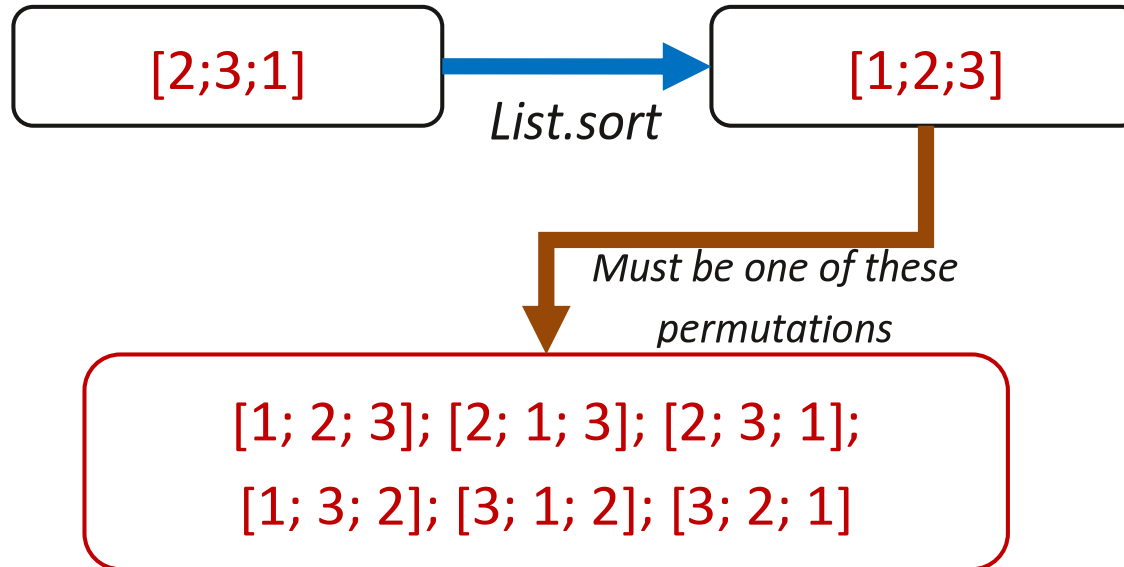


EvilSort just returns an empty list!

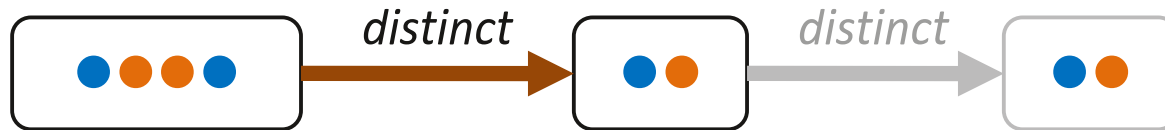
This passes the "commutativity" test!

"Some things never change"

Used to ensure the sort function is good



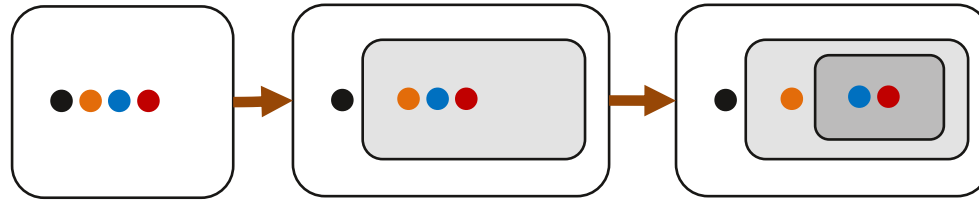
"The more things change, the more they stay the same"



Idempotence:

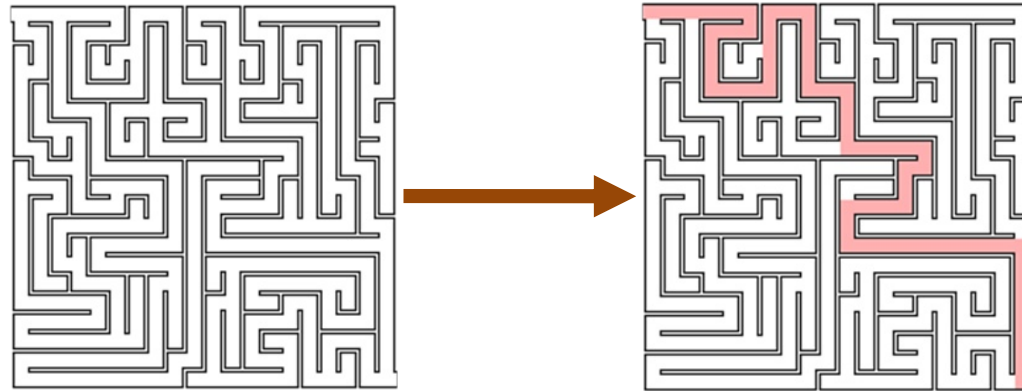
- Sort
- Filter
- Event processing
- Required for distributed designs

"Solve a smaller problem first"



- Divide and conquer algorithms (e.g. quicksort)
- Structural induction (recursive data structures)

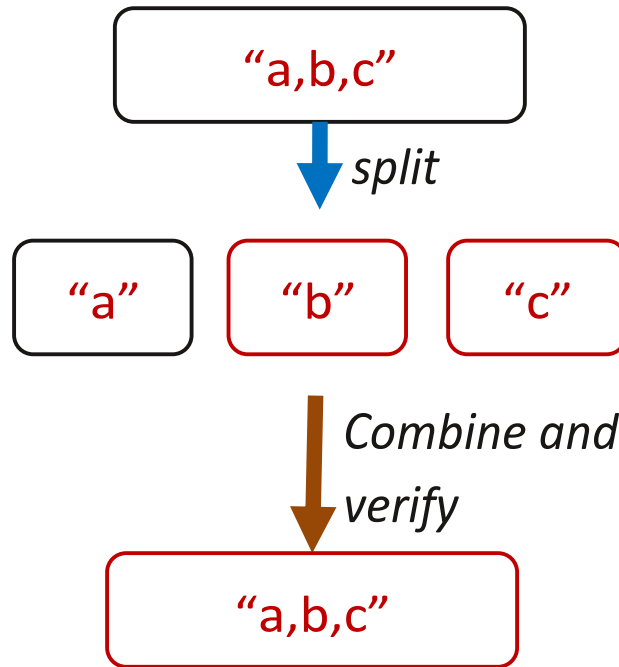
"Hard to prove, easy to verify"



- Prime number factorization
- Too many others to mention!

"Hard to prove, easy to verify"

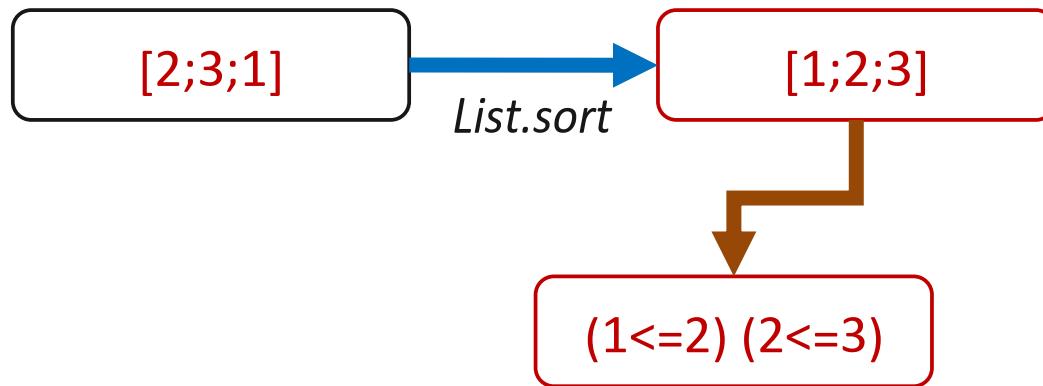
Applied to a tokenizer



To verify the tokenizer, just check that the concatenated tokens give us back the original string

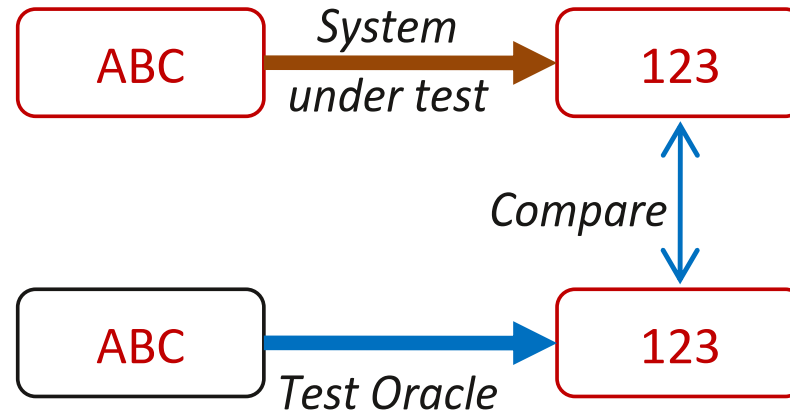
"Hard to prove, easy to verify"

Applied to a sort



*To verify the sort,
check that each pair is ordered*

"The test oracle"



- Compare optimized with slow brute-force version
- Compare parallel with single thread version.

Part V:

Model based testing

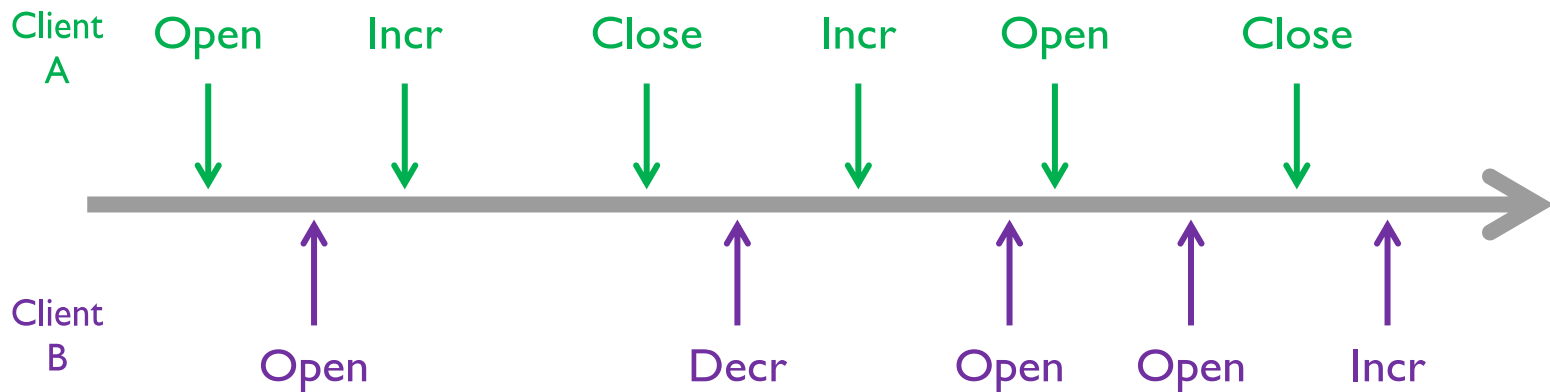
Using the test oracle approach
for complex implementations

Testing a simple database

Four operations: Open, Close, Increment, Decrement

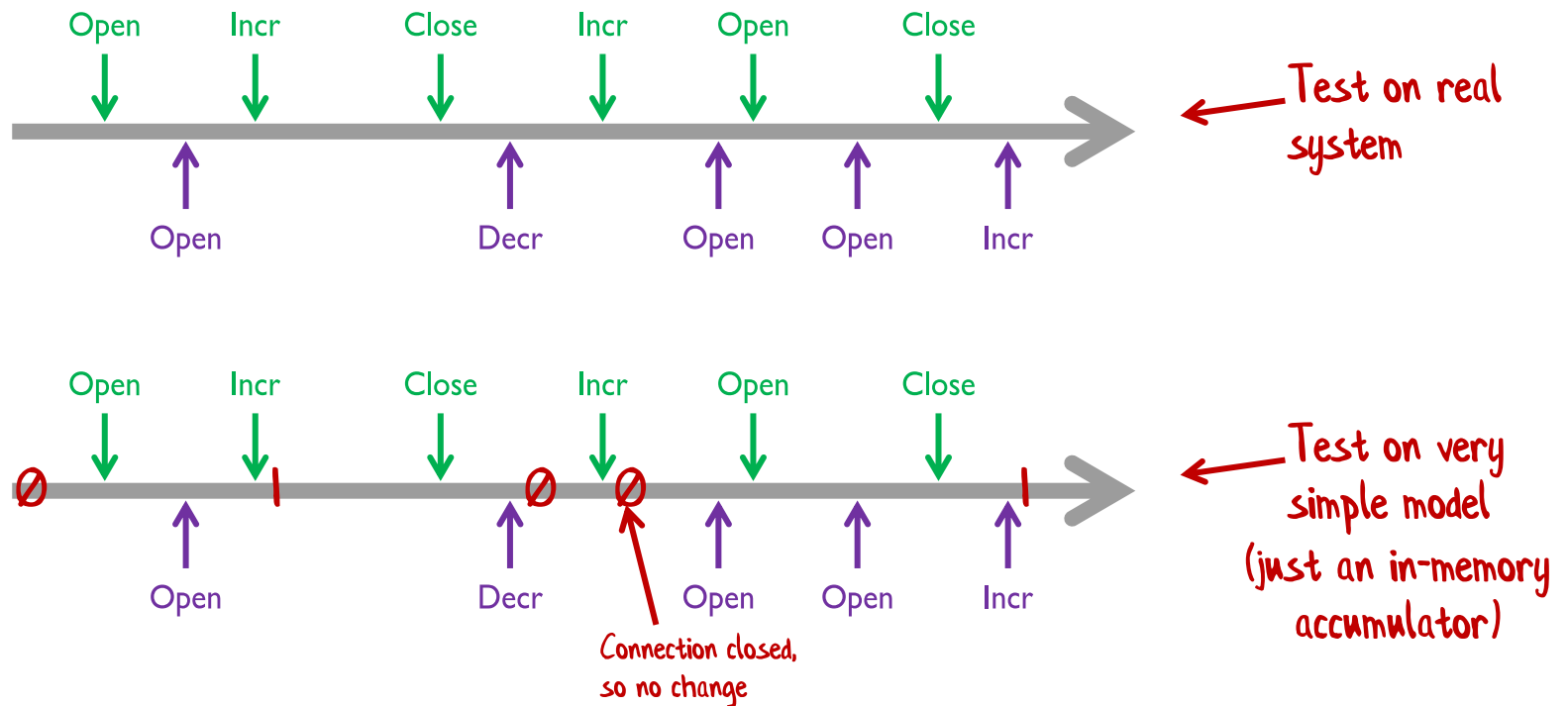
Two clients: Client A and Client B

Let QuickCheck generate a random list of these actions for each client



How do we know that our db works?

Testing a simple database



Compare model result with real system!

Real world example

- Subtle bugs in an Erlang module
- The steps to reproduce were bizarre
 - open-close-open file then exactly 3 parallel ops
 - no human would ever think to write this test case
- Shrinker critical in finding minimal sequence
- War stories from John Hughes at <https://vimeo.com/68383317>

Example-based tests vs. Property-based tests

Example-based tests vs. Property-based tests

- PBTs are more general
 - One property-based test can replace many example-based tests.
- PBTs can reveal overlooked edge cases
 - Nulls, negative numbers, weird strings, etc.
- PBTs ensure deep understanding of requirements
 - Property-based tests force you to think! ☹️
- Example-based tests are still helpful though!
 - Easier to understand for newcomers

Summary

Be lazy! Don't write tests, generate them!

Use property-based thinking to gain deeper insight into the requirements